

Rewind — The Decision Black Box Recorder

A flight recorder for investment decisions. Point it at any position and date, and AI agents reconstruct the **regulator-ready dossier** — who knew what, when, which market data was public at the time, which approvals were signed — in **90 seconds instead of three weeks**. Hackathon build plan v2, July 2026.

BUILD WINDOW 4–5 days · team of 2–4	STACK Vite React 18 + CopilotKit · ASP.NET Core + Agent Framework · Azure AI Foundry	DATA Synthetic vault with planted provenance defects + as-of FRED/Treasury market state
MONEY MOMENT “The approval predates the evidence it cites” — a red arrow across the timeline		

01 · THE PITCH

The firm can finally prove why it did what it did

A regulator (or the internal investment committee) asks: *“On 14 March 2025 the firm added a large NordStar Industrials position the day before its rating downgrade. Reconstruct the decision and demonstrate no material non-public information was involved.”* Today that answer takes a compliance team three weeks of grepping Outlook exports and Teams channels. In Rewind, the Head of Compliance types the position and date, confirms scope at a human gate, and watches agents assemble the who-knew-what-when chain live — vault evidence on one timeline lane, what the

market publicly knew on the other. The gap between those lanes is precisely where red flags live.

Why it's not another finance AI demo: it never recommends, scores, or gates a trade — it looks strictly backward and treats the vault plus the market tape as *evidence*. Bloomberg ASKB and Microsoft Researcher/Analyst look forward and summarize; nobody reconstructs with provenance. And the adversarial judges' favorite property: the money-moment gotcha is a **deterministic timestamp comparison** (`approvalTs < modelEditedTs`) dressed in agent narration — reproducible on stage, never dependent on flaky LLM insight.

The 3-minute demo

0:00–0:20

The inquiry.

Priya pastes the regulator's question. A stopwatch starts; a ghost bar reads “manual baseline: ~3 weeks.”

0:20–0:35

The scope gate.

The Orchestrator parses entity + date window and asks Priya to confirm scope — human-in-the-loop before any evidence is touched. Governed agents, on display.

0:35–1:15

The forensic sweep.

Vault Forensics streams evidence cards: IC minutes, the risk approval, the analyst note, a Teams thread — each with author, role, timestamp. Market Snapshot rebuilds the 14 Mar 2025 tape: curve levels as-of that date, the spread, which news was published *by then*.

1:15–1:45

The black-box timeline assembles.

Two lanes — “what the firm knew” vs “what the market knew” — merge into one strictly-ordered chain ending at the trade.

1:45–2:15

The red flag.

Gap & Red-Flag agent: *“WARNING — the IC decision cites a valuation model last edited 6 hours AFTER the risk approval was signed. The approval predates its own evidence.”* A red arrow draws across the two timestamps. Click the flag: the deterministic rule that caught it is shown, both source docs one click away.

2:15–2:35

The control.

Priya runs a second decision — every check green: “fully defensible.” Proof the system isn't rigged theater; it can exonerate as well as incriminate.

2:35–3:00

Export.

One click renders the paginated dossier, every claim hyperlinked to its source document. Stopwatch stops:
~90 seconds vs 3 weeks.

02 • THE AGENTS

Five agents, one governed pipeline

AGENT	JOB	TOOLS / DATA	MODEL
Reconstruction Orchestrator	Parses the inquiry, plans the sweep, owns the confirm-scope human gate, assembles the dossier	Specialists as in-process function tools; AG-UI entry agent	gpt-5 / gpt-5.4 (medium thinking)
Vault Forensics	Retrieves and time-orders every private artifact: IC minutes, approvals, research notes, Outlook, Teams — with author, role, created/modified timestamps	Azure AI Search hybrid + vector over the synthetic vault	gpt-5-mini
Market Snapshot Reconstructor	Rebuilds the market picture as it was on the decision date — no hindsight	Deterministic C# tool: as-of-date filtering over FRED/Treasury series + publication-stamped synthetic news	gpt-4.1-mini
Timeline Assembler	Merges vault + market evidence into one strictly-ordered event chain; emits the timeline structure the UI renders	Deterministic C# merge/sort — no LLM in the ordering path	— (pure code)

AGENT	JOB	TOOLS / DATA	MODEL
Gap & Red-Flag the payoff	Audits the chain for provenance defects: out-of-order timestamps, unsigned approvals, restricted-list proximity, traded-ahead-of-news gaps	C# rules pass over the deterministic timeline + LLM narration; every flag cites source doc + timestamp	gpt-5-mini

DESIGN RULE

The timeline ordering and every red-flag trigger are **deterministic C# — never LLM output**. The LLM narrates and cites; the rules decide. Show the rule on screen when a flag fires. This is what converts a skeptical judge's "the corpus was rigged" into "the reconstruction logic is real."

03 • ARCHITECTURE

Unchanged plumbing, new payload

Everything below carries over from the original Deal Room design — same stack, same integration pattern, same day-0 gates. Only the agents and the UI payload changed. Starter: CopilotKit monorepo [examples/integrations/ms-agent-framework-dotnet](#) ; per-feature examples at [dojo.ag-ui.com](#).

```
Vite + React 18 (Tailwind · shadcn · TanStack · CopilotKit)
  evidence cards · two-lane black-box timeline · red-arrow flags · scope gate · dossier export
  |
  ▼
Node CopilotKit runtime (~30 lines · HttpAgent → C# endpoint · ExperimentalEmptyAdapter)
  | AG-UI protocol over HTTP + SSE
  ▼
ASP.NET Core (.NET 8) — Microsoft Agent Framework v1.0
  builder.Services.AddAGUI(); app.MapAGUI("/", reconstructionOrchestrator);
  specialists in-process as function tools — each call renders as a live CopilotKit card
  timeline merge + red-flag rules: deterministic C#, out of the LLM's hands
  |
  └─ Azure AI Foundry  AIProjectClient(endpoint, cred).AsAIAgent(...) · GPT-5.x / GPT-4.1
  └─ Azure AI Search   Basic tier · the synthetic vault (hybrid + vector, timestamp metad
  └─ Cosmos DB         free tier · inquiry log + dossier audit trail (audit IS the produc
```

Multi-agent *workflow* streaming over AG-UI remains Python-only (.NET “coming soon”). One AG-UI agent orchestrates in-process; specialists surface as function-tool calls → live generative-UI cards. Judges watch five agents work without betting on prerelease bits.

04 • DATA PLAN

A vault authored to be cross-examined

The planted needles (3–4 sharp ones, ranked — never 15)

- **The gotcha:** risk-approval signature timestamped *before* the valuation model file it cites was last edited
- Analyst research note dated *after* the trade (hindsight-fabrication flag)
- Teams thread naming a contact who appears on a planted restricted list (MNPI-proximity flag)
- IC minute naming attendees + rationale (the clean backbone of the chain)
- **The control:** a second, fully documented decision that renders an all-green dossier — Rewind must be able to exonerate

Market side — honest framing

FRED/Treasury series filtered as-of the decision date give “what was public when”; synthetic news carries publication timestamps. **Narrate it as observation-date filtering, not point-in-time vintage data** — a finance-literate judge will poke an overclaim. The fusion is the two timeline lanes: what the firm privately knew vs what the market publicly knew. An information gap between the lanes *is* the compliance finding.

Retrieval

Azure AI Search Basic (~\$75/mo prorated, inside the \$200 credit), Import-and-vectorize wizard, `text-embedding-3-small` at index *and* query time. Rich timestamp/author/role metadata on every doc — the forensics live in the metadata.

05 • DAY 0 GATES

Unchanged — do these before writing any code

- ☐ **Register for GPT-5 access** (aka.ms/openai/gpt-5) — gates deployment. GPT-4.1 wired as fallback.
- ☐ **Entra ID auth** — no API-key path on the Foundry Projects client. `az login` for every teammate + RBAC on the project.
- ☐ **Right RBAC roles** — Foundry User (GUID 53ca6127-db72-4b80-b1b0-d745d6d5456d) for `AIProjectClient`; Cognitive Services OpenAI User for `AzureOpenAIClient`. GUIDs in Bicep; ~10 min propagation.
- ☐ **Region check** — Responses API + File Search + Code Interpreter coverage before provisioning.
- ☐ **Quota bump** — concurrent agent fan-out hits 429s on default TPM; chatty agents on gpt-4.1-mini.
- ☐ **Pin prerelease NuGet versions** for the AG-UI hosting packages; never upgrade mid-hackathon.
- ☐ **Clone the starters:** CopilotKit monorepo .NET example + `Azure-Samples/ai-chat-quickstart-csharp`.

06 • BUILD SCHEDULE

Day by day

DAY	DELIVERABLE	DETAIL
0 (½ day)	All gates cleared	Checklist above + hello-world <code>azd up</code> so deployment risk dies early. Vanilla starter streaming locally.
1	Evidence corpus + as-of market tool	Author the vault with timestamp-consistent needle chains; index in AI Search; acceptance test: the mis-ordered approval retrieves with intact metadata . Build the C# as-of FRED filter. Parallel: React shell.
2	Agents end-to-end	Vault Forensics + Market Snapshot + deterministic Timeline Assembler + red-flag rules pass. Full dossier object from a console test, red flag firing on the planted needle.

DAY	DELIVERABLE	DETAIL
3	The timeline UI	The big UI investment (no library shortcut for this): two-lane black-box timeline, evidence cards streaming via CopilotKit, the red-arrow animation, scope gate. Legibility in 90 seconds is the bar.
4	Dossier + control + deploy	Dossier export (with pre-rendered fallback — mandatory, not optional), the all-green control decision, audit log in Cosmos, <code>azd up</code> to Container Apps.
5	Polish + rehearse	Stopwatch + baseline bar; rank flags by severity; rehearse the 3-minute demo three times.

Cut lines (drop in this order if behind)

1. Live PDF export → pre-rendered dossier PDF opened on click (the timeline stays live)
2. Restricted-list / MNPI-proximity flag → keep only the timestamp gotcha + hindsight-note flags
3. All-green control decision → narrate it instead of running it
4. Azure deploy → demo from localhost against live Foundry + AI Search

The guaranteed-demoable core: Vault Forensics + deterministic timeline + the one timestamp red flag, streamed as agent cards. That alone lands the story and every hard constraint.

07 • DEPLOYMENT

Azure topology (unchanged)

PIECE	HOST	WHY
ASP.NET Core agent API	Azure Container Apps (0.5 vCPU / 1 GiB)	SSE streams cleanly through Envoy; managed identity built in
Node CopilotKit runtime	Container Apps , same environment	Intra-environment hop; SSE-safe

PIECE	HOST	WHY
Vite static build	Static Web Apps Free — or serve <code>dist/</code> from a container	Free hosting + auto HTTPS
Evidence index	AI Search Basic	Hybrid + semantic retrieval with metadata filters
Inquiry + audit log	Cosmos DB free tier	Lifetime free (1000 RU/s, 25 GB); audit is the product

- Avoid App Service for the API — Windows plans buffer SSE through IIS/ARR. Container Apps on Linux/Envoy avoids the bug class.
- `minReplicas: 1` on both server apps; heartbeat every ~20s (ingress idles out at 230–240s).
- `DefaultAzureCredential` promotes `local` → managed identity with no code change; RBAC takes ~10 min to propagate (“works locally, 403 in cloud” = wait, don't debug).
- First green deploy: budget a half-day; then `azd deploy <service>` in 1–3 min.

08 • RISK REGISTER

What the adversarial judges flagged

RISK	MITIGATION
Timeline UI illegible under 90 seconds — the gotcha lands flat in a wall of JSON	Day 3 is dedicated to the timeline component; two lanes max; red-arrow animation rehearsed
In-browser paginated, hyperlinked PDF is a classic 1–2 day time-sink	Pre-rendered fallback is <i>mandatory</i> ; live export stays one click, cut first
“Rigged corpus” skepticism — needles were authored to be found	Show the deterministic rule when a flag fires; run the all-green control decision live
As-of reconstruction overclaim (observation-date filter ≠ point-in-time vintage)	Honest narration: “what was published by then,” never “vendor-grade PIT data”

RISK	MITIGATION
eDiscovery/GRC genre adjacency (Relativity, Archer)	Differentiate on the market-fusion lane + visible agent orchestration — neither exists in GRC tooling
Sounds like trade-timing analysis if narrated carelessly	Script stays on “what was public vs private” — no buy/sell language anywhere in UI or narration
Foundry setup gates (GPT-5 registration, Entra-only auth, 429s, prerelease NuGets)	Day 0 checklist; GPT-4.1 fallback; quota bump; pinned versions